

AURIX: Backend & Infrastructure Engineer (Bhavya) - Master Workflow Guide

Welcome to your master workflow, **Bhavya**! As the Backend & Infrastructure Engineer for AURIX, you are the "Traffic Cop," the "Vault," and the Nervous System of the platform. You bridge the gap between the client interfaces (React Dashboard & VS Code Extension) and the heavy AI Worker Node.

This guide provides a comprehensive, step-by-step roadmap to build your component from scratch to a full production deployment using a **Zero-Cost Tech Stack**.

🔧 The Tech Stack Arsenal (Zero-Cost)

Technology	Purpose in AURIX
Node.js & Express.js	Core API Gateway & routing logic.
Supabase	PostgreSQL (DB), Supabase Auth, Object Storage (.zip files), and Vector DB (pgvector).
Upstash (Redis)	Message Queuing (LPUSH) and API Rate Limiting.
Hugging Face	Free Inference API or local Transformers.js for AI Embeddings (RAG).
Render / Railway	Free-tier web hosting for the Node.js API Gateway.

Multer & node-cron	Multipart form parsing for zip uploads and Cron jobs for storage sanitization.
-------------------------------	--



Phase 1: Infrastructure Provisioning & Foundation

Your first goal is to set up the foundational cloud infrastructure and database schemas.

1. Cloud Services Setup

- ☐ **Supabase (Free Tier)**: Create a new project to provision PostgreSQL, Supabase Storage (S3-compatible), and Supabase Auth.
- ☐ **Upstash (Serverless Redis)**: Create a Redis database for task queuing and rate limiting.

2. Database Schema Implementation

Execute SQL in Supabase to create the core tables:

- **USERS**: Managed largely by Supabase Auth (linking UUIDs).
- **PROJECTS**: `id`, `user_id` (FK), `name`, `repository_url`, `created_at`.
- **SCANS**: `id`, `project_id` (FK), `status` (PENDING, SCANNING, COMPLETED, FAILED), `storage_path`, `total_findings`, `verified_findings`, `timestamps`.
- **VERIFIED_VULNERABILITIES**: `id`, `scan_id` (FK), `file_path`, `line_number`, `severity`, `vulnerability_type`, `ai_confidence_score`, `ai_reasoning`, `wargame_status`, `poc_script`, `patch_code`, `is_resolved`.

3. Authentication ("The Vault")

- ☐ Configure Supabase Auth (Email/Password or GitHub OAuth).
 - ☐ Implement middleware in Node.js to validate secure session tokens (JWTs).
 - ☐ Implement Row Level Security (RLS) in Supabase or Backend validation so users can *only* query their own scans.
-

🔧 Phase 2: Core API Gateway & Unblocking Teammates

CRITICAL BLOCKER ALERT: You block Bhumika (Frontend) and Divyanshi (VS Code). Your immediate priority is to deploy **Mock Endpoints** so they can start building.

1. The "Mock" Unblocker (Do This First!)

- ☐ Initialize a Node.js Express server.
- ☐ Create mock endpoints for `POST /api/scans/github` and `POST /api/scans/upload` that return fake `scan_ids` and `202 Accepted`.
- ☐ Create a mock `GET /api/scans/{scan_id}` that returns a hardcoded JSON of vulnerabilities (based on the `verified_report.json` schema).
- ☐ Deploy this mock server immediately to Render or Railway and share the URL with the team.

2. Core Ingestion Routes (Real Logic)

Once teammates are unblocked, replace mocks with real logic:

- ☐ **POST /api/scans/github:** Save project to DB, set status to `PENDING`, push the GitHub URL to the Upstash Redis Queue (`LPUSH`), and immediately return `202 Accepted`.
- ☐ **POST /api/scans/upload:** Use `multer` to handle `multipart/form-data`. Safely stream `.zip` files (~10MB limit)

directly to Supabase Storage using `@supabase/supabase-js`. Create DB record, push storage link to Redis, return `202 Accepted`.

- ☐ **GET /api/scans/{scan_id}**: Look up scan in DB. Return current status (e.g., `SCANNING`) or the final `VERIFIED_VULNERABILITIES` if `COMPLETED`.

⚠ **Asynchronous Processing is Mandatory:** Never await the AI scan in your HTTP POST request. It takes minutes and will timeout. Return `202 Accepted` immediately after pushing to Redis.

Phase 3: The AI Upgrade - RAG Vector Infrastructure

You are responsible for the infrastructure that allows the AI agents to query OWASP guidelines and past CVEs accurately.

1. Vector Database Setup

- ☐ Enable the `pgvector` extension in your Supabase PostgreSQL database (`CREATE EXTENSION vector;`).
- ☐ Design and create tables to store high-dimensional vector embeddings (e.g., a `knowledge_base` table with a `vector` column).

2. Data Seeding Script (`seed_rag.js`)

- ☐ Write a standalone Node.js script to ingest raw security docs (OWASP Cheat Sheets, PoC templates, Patch Code).
- ☐ Use a free Hugging Face model (like `all-MiniLM-L6-v2` via `Transformers.js`) to convert these markdown chunks into vector embeddings.
- ☐ Push the text and their embeddings into the Supabase vector table.

3. Similarity Search API (Threat Intel Endpoint)

- ☐ Create an internal API endpoint (or Supabase RPC function) for Divyansh's LangGraph node to call.
 - ☐ **Input:** Search query (e.g., `{"query": "python.lang.security..."}`).
 - ☐ **Processing:** Convert query to an embedding, perform cosine similarity search against `pgvector`.
 - ☐ **Output:** Return a JSON array of the Top 2 or 3 most relevant markdown text chunks in **under 500ms**.
-

Phase 4: Orchestration, Security, & Optimization

Connect the final pieces and ensure the system is secure and cost-effective.

1. Internal Webhook (AI Worker to Backend)

- ☐ Create an internal `POST` route (e.g., `/api/internal/webhook/scan-complete`).
- ☐ Divyansh's AI Worker will `POST` the massive JSON payload to this route when a scan finishes.
- ☐ Update the `SCANS` table status to `COMPLETED` or `FAILED` and populate the `VERIFIED_VULNERABILITIES` table.

2. Rate Limiting

- ☐ Implement Redis-backed API Rate Limiting (using `express-rate-limit + rate-limit-redis`).
- ☐ Restrict users to a maximum number of scans per hour/day to protect computationally expensive AI resources. Return `429 Too Many Requests` if exceeded.

3. Automated Data Sanitization

- ☐ Implement a cron job (using `node-cron`) to run every hour.
 - ☐ Automatically delete the `.zip` payload from Supabase Storage once a scan is marked `COMPLETED` or `FAILED`. This prevents massive storage bloat (staying under the 1GB free tier limit) and protects user source code privacy.
-

🚫 "Do Not Miss" Constraints Checklist

- ☐ **Auth Handshake:** Frontend has exact routes for login/signup; all GET/POST routes are secured with JWTs.
- ☐ **Robust Zip Handling:** Streams are handled without crashing your lightweight Node.js server memory.
- ☐ **Data Privacy:** Source code (`.zip`) is deleted post-scan. Only vulnerability metadata is kept.
- ☐ **Compute Protection:** Redis rate limiter is active and prevents queue flooding.
- ☐ **RAG Latency:** Vector retrieval happens in < 500ms using high-quality embeddings.